Performance Analysis of Sparse Matrix-Vector Multiplication (SpMV)

Michael Bernasconi
ID Student: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication
Department of Information Engineering and Computer Science
University of Trento

***Abstract*——-**
***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

The Sparse Matrix-Vector Multiplication (SpMV) operation, defined as $y = Ax$, costitutes a critical computational kernel for High-Performance Computing (HPC) and scientific simulations. Due to its low arithmetic intensity, quantifiable as $2 \times nnz$ total operations, SpMV is an inherently memory-bound operation. Its effiency is further hindered by the irregulare nature of memory accesses, which are strictly dependent on the matrix sparsity structure. This study analyzes the impact of storage formats and parallelization stategies on the NVIDIA Ampere architecture, comparing custom CUDA implementations (COO, CSR, and CSR-Vector) with the highly optimized cuSPARSE library. The investigation evaluates performance through key metrics: kernel execution time, throughput (GFLOPS), and effective Bandwidth (BW). Beyond a standard comparative evaluation, this work examines the link between data topology and performance, analyzing how parallel computing efficiency is influenced by memory organization and cache hit/miss rates. The analysis demonstrates how the structural characteristics of matrices from different application domains-raning from fluid dynamics to socail network graphs-decisively influence the overall operation speed and hardware utilization. The objective is to provide a comprehensive experimetnal study that clarifies under which structural conditions specific formats or kernels should be prederred to mitigate the instrinsic bottlenecks of sparse computation.

II. Methodology

Formats tested, CPU/GPU implementations, validation method, measurement methodology, and hardware/software environment.

- **Hardware & Software Environment:**
  - Specify the target architecture (e.g., NVIDIA A30 GPU, Ampere), CUDA toolkit version, compiler flags (e.g., `-O3`, `-arch=sm_80`), and OS.
  - Mention the use of Valgrind –¿ (only cpu cache, beacuse GPU the tools are blocked until the second deliverable)
- **CPU Baseline & GPU Kernels:**
  - *CPU Baseline:* A straightforward sequential or OpenMP implementation, explicitly stating it is used for validation and is not aggressively optimized[- 12, 38].
  - *GPU Formats:* (PSEUDOCODICE) Briefly describe the tested formats (e.g., COO, CSR, CSR-Vector, cuSPARSE).
  - *Kernels Tested:* Explain your implemented kernels: Base COO, Base CSR, and your **Optimized CSR**. Detail how you use shared memory to reduce the cost of accessing global memory and how warp-level primitives improve load balancing.
- **Measurement Methodology:**
  - *Consistency:* To ensure numerical consistency between write-only formats (CSR) and atomic-accumulate formats (COO), state that an explicit output vector reset was included within the benchmark loop.
  - *Variability Mitigation:* Explain that runtime is isolated strictly to kernel execution, ignoring one-time setup costs like file parsing. To mitigate shared environment noise, each benchmark performs 5 independent runs, reporting the best-of-5 (minimum execution time) as a justified stable statistic.
- **Performance Metrics (Formulas):**
  - State the formula for computational throughput (GFLOP/s), defining the standard estimate of 2 floating-point operations per non-zero (NNZ) element.
  - Define the formula for Effective Bandwidth (GB/s).
  - State the Theoretical Peak Bandwidth of the target GPU (e.g., ∼933.1 GB/s for the A30) as the theoretical upper bound.
  - Provide formulas for Cache Hit/Miss rates as extracted from the profiler.
- **Correctness & Validation:**
  - Describe the validation function `validate_results(float *y_cpu, float *y_gpu, int M)`.

- Since `Float32` is used, explain the tolerance-based comparison method (e.g., $\epsilon = 1e-3$). Implement a `for` loop over the entire vector: if $|y\_cpu[i] - y\_gpu[i]| > \epsilon$, print "FAIL", otherwise "PASS".
- *Theoretical Explanation:* Explain why validation depends on the representation method. In parallel architectures (GPUs), the thread execution order for additions (e.g., via `atomicAdd`) is non-deterministic. Because floating-point arithmetic is non-associative, different summation orders inevitably generate slightly different rounding errors.

III. Dataset

The experimental evaluation is conducted on a selection of 10 sparse matrice from *SuiteSparse Matrix Collection*, following the benchmark suite proposed by Chu et al. [1]. These datasets were specifically chosen to represent a wide variety of applciation domains and structural characteristics.

TABLE I
STRUCTURAL CHARACTERISTICS OF THE SELECTED SUITESPARSE MATRICES

Matrix Name	Rows (M)	Cols (N)	NNZ	Avg. NNZ/R	Std. Dev. Rows
ASIC_680ks	682,712	682,712	1,693,767	2.48	4.16
bone010	986,703	986,703	47,851,783	48.50	7.62
boyd2	466,316	466,316	1,500,397	3.22	194.91
FullChip	2,987,012	2,987,012	26,621,983	8.91	1,806.80
Ga41As41H72	268,096	268,096	18,488,476	68.96	105.39
ldoor	952,203	952,203	42,493,817	44.63	14.77
rajat31	4,690,002	4,690,002	20,316,253	4.33	1.11
Rucci1	1,977,885	109,900	7,791,168	3.94	0.34
Si41Ge41H72	185,639	185,639	15,011,265	80.86	126.97
webbase-1M	1,000,005	1,000,005	3,105,536	3.11	25.35

A. Input Generation and Precision

The test utilize single precision (`Float32`), whic is the standard for high performance computing on GPUs. The dense vector x ($N \times 1$) is generated using the `fill_random_vector` function, which initializes the pseudo-random generator with a fixed seed (`srand(42)`). Values are normalized within the range $[0, 1]$ using the ratio `rand()/RAND_MAX`. This approach ensures the reproducibility od results between CPU and GPU, isolating the perfomance analysis solely to the strucutral characteristics of the matrices and the efficiency of the kernels.

IV. Results

Plots/tables for runtime and FLOPS/GFLOP/s; optional memory/cache indicators. *(Note: Visual data presentation only; formulas are established in Methodology).*

- **Throughput & Runtime:** Include bar charts comparing GFLOP/s across the 10 matrices for Base CSR, Base COO, Opt CSR, and cuSPARSE. Include kernel-time plots displaying the measured temporal variability (e.g., standard deviation or min-max bounds).
- **Bandwidth Plots:** Create Effective Bandwidth (GB/s) charts. Add a horizontal line representing the Theoretical Peak Bandwidth (e.g., ∼933.1 GB/s) to visually demonstrate how close the implementations are to being memory-bound.
- **Memory/Cache Indicators:** Provide charts or tables (using data from `valgrind`) showing cache hit/miss behavior and global memory accesses.
- Nonostante le ottimizzazioni del kernel, il TTS rimane dominato dal caricamento I/O per le matrici meno dense

V. Discussion

Interpret the results in terms of format properties, matrix structure, and memory behavior. Connect the measured behavior to algorithmic techniques and matrix characteristics.

- **Load Balancing & Matrix Structure:** Do not simply state "kernel X is faster". Explain *why*. Discuss how row-length regularity and variance affect performance. For instance, precisely explain under which structural conditions (e.g., highly irregular matrices) CSR-Vector outperforms CSR-Base by mitigating load imbalance.
- **Memory Behavior & SM Utilization:** Address how close the results are to the memory bound and what evidence supports this. Use the `valgrind` profile cache for cpu.
- **The cuSPARSE Advantage:** Analyze the performance gap with cuSPARSE, theorizing its use of adaptive load-balancing techniques, dynamic partitioning, or undocumented heuristics tailored to specific matrix regimes.

VI. Conclusion

Main lessons learned, limitations, and practical takeaways.

- **Main Lessons Learned:** Summarize which observations are intrinsic to the mathematical structure of the matrix itself, and which are artifacts of specific implementation choices.
- **Practical Takeaways:** Clearly state the optimal use-cases. Explain exactly under which matrix characteristics (e.g., low NNZ/row, high variance) and memory-access conditions a specific kernel or format ceases to be advantageous and another should be preferred.
- **Limitations:** Briefly mention study limitations (e.g., exclusively testing `Float32` arithmetic or specific Ampere architecture constraints).

References

[1] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, X. Wang, and C. Hu, "Efficient Algorithm Design of Optimizing SpMV on GPU," in *Proc. 32nd Int. Symp. High-Perform. Parallel Distrib. Comput. (HPDC '23)*, 2023, pp. 243–256. doi: 10.1145/3588195.3593002.